LAB 5 · AN INTRODUCTION

Skills

A folder with a markdown file in it.
The interesting part is when it gets read.



FOR	Anyone who has not written one.
READ IT	Before Sunday morning. Ten minutes.

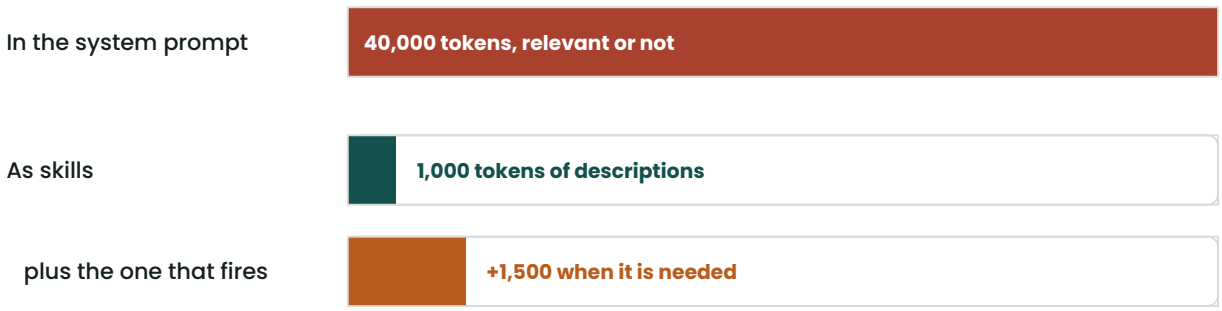
What it actually is

A skill is a folder containing a file called SKILL.md. That file has a name, a one-line description, and then instructions written in plain English. That is the entire format.

The deflation is deliberate, because the format is not the idea. The idea is **when** the instructions get read.

Everything a model is told costs tokens, and because the API is stateless you pay for all of it again on every single request. So the obvious approach - put all your procedures in the system prompt - stops working almost immediately. Twenty procedures at two thousand tokens each is forty thousand tokens on every call, whether the request has anything to do with them or not.

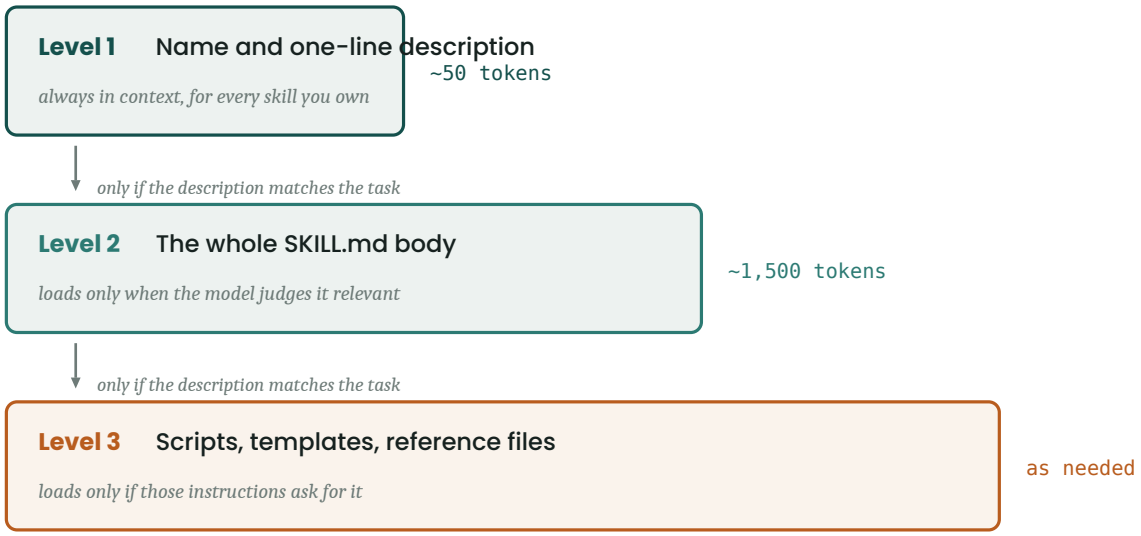
Twenty procedures, on every single request



Same twenty procedures either way. The saving is not the point by itself - the point is that it lets you have twenty at all.

Skills solve that with **progressive disclosure**. Only the name and the one-line description are always present. The model reads those, decides whether a skill is relevant to what it has been asked, and loads the rest only if it is.

What is loaded, and when



The model does the routing. You are not writing an if-statement.

The consequence nobody expects

Because level one is all the model sees when it decides, **the description line is the most important line in the file**. It is doing all the routing work, and it is the part people write last and least carefully.

> **THE FAILURE YOU WILL HAVE, THE FIRST TIME**
>
> A vague description never fires. The skill sits there, perfectly written, and is never read - and because nothing errors, you conclude that skills do not work. Write the description as "Use this when the user asks about X, Y or Z", and name the nouns that should trigger it.

Example 1 - Five people, five formats

The problem skills were built for, and the one most teams have without noticing.

Your team writes incident reports. There is a template for them in Confluence. Nobody opens it. So five engineers write five different shapes of report, the severity field means something different to each of them, and the monthly review is an exercise in archaeology.

This is not a knowledge problem. Everyone knows how an incident report should look. It is a *consistency* problem, and asking people to try harder has never once fixed one.

YOU	**Write up last night's outage. Logs are in the thread.**
AGENT	I have an incident-report skill available - loading it.
RUNS	`reads skills/incident-report/SKILL.md`
AGENT	Writes the report in the house format: timeline in UTC with one line per event, impact stated in customers affected rather than services affected, severity from the fixed four-value scale, contributing factors separated from root cause, and actions with named owners.

The same shape, every time, from everyone. That consistency is the product - it is what makes the monthly review possible, and it is what you could never get by sending round a template.

> **WHY THIS BEATS A BETTER PROMPT**
>
> You could paste the template into a prompt each time. The difference is that a skill is a file in a repository: it gets reviewed, versioned, improved, and rolled back. When the format changes, it changes in one place for everyone.

Example 2 - The rules that are expensive to re-explain

Domain knowledge that lives in one person's head, written down once.

Every field has rules that are obvious to the people who have been burned by them and invisible to everyone else. A data team knows that an identifier column must never be summed, even when it is numeric. A finance team knows which of two date fields is the one that matters for a cut-off. A clinical team knows which codes are never reported without a denominator.

These are the rules you explain to every new starter, in person, twice. A skill is where they go instead.

YOU	**Profile this CSV for me.**
AGENT	Loading the csv-profiler skill.
RUNS	`reads skills/csv-profiler/SKILL.md`
AGENT	Shape, then a column table with an inferred type for each. Note that order_ref is numeric but is an identifier, so it is typed as such and excluded from the summary statistics. Two quality flags: a 7% null rate in region, and inconsistent capitalisation in channel.

The useful detail is the one in the middle. Nothing in the data says *order_ref* is an identifier - it is a column of integers. The skill said so, because somebody who had once averaged a column of order numbers wrote it down.

What a skill is good for

Procedure	A sequence that must be followed the same way every time. Reviews, reports, handovers, checklists.
House format	An output shape your team has agreed on and keeps failing to apply consistently.
Hard-won rules	The gotchas that are obvious only to people who have already been caught by them.
Tool conventions	How your organisation uses a thing, as distinct from how the thing works.

Where it goes wrong

Three failures, in the order you will meet them.

It never fires.	The description was too vague, or used different words from the ones people actually type. The skill is invisible and nothing tells you. Test the trigger, not just the content: ask three differently-worded questions and check it loads.
It fires when it should not.	The description was too broad, so a csv-profiler skill loads when someone asks to list the files in a folder. Wasted tokens and a confused answer. Test this too - ask something adjacent and check it stays quiet.
It is ignored once loaded.	Instructions that suggest get ignored; instructions that specify get followed. "Consider using a table" is decoration. "Output a markdown table with exactly these five columns" is a procedure.

THE ONE THAT IS NOT A USABILITY PROBLEM

A skill is a set of instructions that a model will follow, stored in a file. If someone can commit a file to your repository, they can change what your agents do - and a skill that fetches content from an external URL can have its instructions changed by someone outside your organisation entirely. Review skills the way you review code, because that is what they are.

What to take into the lab

ONE

A skill is a folder with a markdown file in it. The format is not the idea; **when it gets read** is the idea.

TWO

The description does the routing and the body does the work. Most first skills fail at the description.

THREE

The product is consistency. A model that writes a good report is useful; one that writes the *same* report every time is something you can give to a team.

In the lab you will write a skill, watch a model discover and load it, and then swap with your partner - because the fastest way to learn what a good description looks like is to watch yours fail to fire on somebody else's phrasing.